Calculatrice ou jeu interactif (Python)

Robotique – Grade 10 – Plan de cours de 2 heures

Enseignant : Francis Ki | Semaine du 23 au 27 novembre 2026

1 Informations générales

Titre	Calculatrice ou jeu interactif : un programme complet
Classe / effectif	Grade 10, 11 à 20 élèves, niveaux hétérogènes, élèves à besoins particuliers
Durée	2 heures (120 min)
Lieu	Salle informatique, un poste par élève
Matériel et logiciels	— Un poste par élève avec l'éditeur **Mu** en mode *Python 3* (Vérifier, Exécuter, Débogueur). — Vidéoprojecteur ou tableau interactif avec Mu projeté. — Un dossier partagé contenant les fichiers de départ `.py` des trois niveaux et la grille d'évaluation du projet. — Inspiration : unité « Putting it all together » du cours TeachComputing (binômes, codage en direct, exemples commentés).
Pré-requis	Séances précédentes : variables, types, saisie, conditions, boucles `for` et `while`.

Préparation avant le cours (environ 45 min)

1. Préparer un exemple fini de calculatrice et un exemple fini de jeu (à montrer en démonstration).
2. Préparer les fichiers de départ : `niveauA.py`, `niveauB.py`, `niveauC.py`.
3. Préparer la grille d'évaluation du projet (annexe B) et le formulaire de sortie (annexe G).
4. Prévoir la séance de décembre : « Introduction au HTML », qui suivra l'évaluation.

2 Objectifs d'apprentissage

Objectifs du programme

— Concevoir un programme fonctionnel.

— Corriger les erreurs détectées.

À la fin de la séance, l'élève est capable :

— de décomposer un programme en étapes (pseudo-code) avant de le coder ;

— d'assembler variables, conditions et boucles dans un programme complet ;

— de distinguer et de corriger une erreur de syntaxe, d'exécution et de logique.

Compétences numériques : planifier, tester par un jeu d'essais, lire un message d'erreur, enregistrer des versions.

3 Déroulé minuté (120 min)

Minutes	Étape	Contenu	Organisation
0 – 10	Accroche	Démonstration d'une calculatrice et d'un jeu	Classe entière
10 – 30	Mini-leçon	Du pseudo-code au programme ; les trois types d'erreurs	Classe entière
30 – 60	Pratique guidée	Pseudo-code collectif, puis première opération en binômes	Collectif puis binômes
60 – 95	Pratique autonome	Trois niveaux de besoin	Individuel sur poste
95 – 110	Mise en commun	Présentation des programmes	Classe entière
110 – 120	Exit ticket	Trois questions sur poste	Individuel sur poste

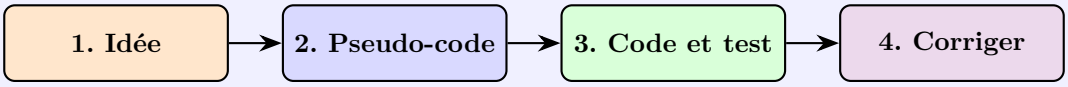
1. Accroche : deux programmes à découvrir — 0 – 10 min

1. (5 min) Faire fonctionner une calculatrice (menu, boucle, quitter) et un jeu de devinette. *Quelles variables, conditions et boucles y voyez-vous ?*
2. (5 min) Les élèves choisissent leur projet : calculatrice ou jeu. Question-problème : **« Comment transformer une idée en programme qui fonctionne et qui ne plante pas ? »**

2. Mini-leçon : planifier, tester, déboguer — 10 – 30 min

Support : annexe A. Codage en direct. Quatre temps de 5 min.

Temps 1 : du pseudo-code au code.



Pseudo-code d'une calculatrice

```
# repeter jusqu'a ce que l'utilisateur quitte
#   afficher le menu
#   demander l'operation
#   si quitter : arreter
#   demander deux nombres
#   selon l'operation : calculer
#   afficher le resultat
```

Temps 2 : les trois types d'erreurs.

Type	Exemple	Comment la trouver
Syntaxe	Deux-points oublié, parenthèse non fermée	Mu la signale avant ou à l'exécution
Exécution	Division par zéro, `int("abc")`	Le programme s'arrête avec un message (*Traceback*)
Logique	Mauvaise condition, résultat faux	Aucun message : il faut tester

Temps 3 : lire un message d'erreur. Montrer un *Traceback* : la dernière ligne dit le type d'erreur, l'avant-dernière donne la ligne concernée.

Temps 4 : le jeu d'essais. Avant de dire « ça marche », tester des cas normaux, des cas limites (0, négatif) et des saisies absurdes.

Différenciation et besoins particuliers

Fournir le pseudo-code sous forme d'étiquettes à ranger dans l'ordre. Utiliser les couleurs de l'annexe A pour distinguer les trois erreurs. Autoriser le travail en binôme sur la planification.

3. Pratique guidée — 30 – 60 min

Fiche : annexe B. Programmation en binômes (pilote et navigateur, rôles échangés à mi-temps).
3a. Pseudo-code collectif (12 min). Écrire ensemble le pseudo-code de la calculatrice, puis coder le menu et la première opération (addition).
3b. Première opération en binômes (18 min). Chaque binôme code, selon son projet, soit l'addition et la soustraction avec le menu, soit la partie « choisir un nombre secret et le comparer ». Tester avec trois entrées.
Étayage progressif de l'aide.

Niveau d'aide	Contenu	Qui intervient
Aide 1	Le pseudo-code du 3a et les modèles de code des séances précédentes	Le binôme seul
Aide 2	Questions guides : *Quelle est la prochaine étape du pseudo-code ? Quelle structure (condition ou boucle) faut-il ?*	Le binôme avec la carte d'aide
Aide 3	L'enseignant écrit la première ligne de code avec le binôme	L'enseignant

Différenciation et besoins particuliers

Insister : coder une étape, tester, puis ajouter la suivante. Laisser ouverte la liste des modèles de code. En cas de blocage prolongé, passer au pseudo-code sur papier.

4. Pratique autonome différenciée — 60 – 95 min

Consigne commune : chaque élève ouvre le fichier de son groupe, le renomme avec son prénom et travaille seul sur son poste. L'enseignant passe d'abord auprès du groupe A pendant 10 min.

Groupe	Tâche sur poste	Annexe
A : en dessous du niveau	Fichier `niveauA.py` : calculatrice à deux opérations dont les lignes essentielles sont à compléter, avec le pseudo-code fourni et une banque de mots. Tester trois calculs.	Annexe C
B : au niveau	Programmer une calculatrice à quatre opérations avec menu en boucle, gestion de la division par zéro et option « Quitter », ou un jeu de devinette complet. Fournir un jeu d'essais.	Annexe D
C : au-dessus du niveau	Réaliser un projet complet et structuré : calculatrice avec fonctions (`def addition(a, b):`, etc.) et historique des calculs dans une liste, ou jeu avec rejouer et meilleur score. Gérer les saisies invalides avec `try` et `except`.	Annexe E

Plan B si un élève termine tôt : tester le programme d'un camarade en cherchant à le faire planter, et noter les bogues trouvés.

5. Mise en commun : présentation des programmes — 95 – 110 min

1. (2 min) Retour au calme.
2. (9 min) Trois présentations courtes (3 min), avec valorisation :
   - **Groupe A** : un élève montre ses calculs et explique une ligne. Valoriser la compréhension de son programme.
   - **Groupe B** : un élève présente son jeu d'essais et un bogue corrigé. Valoriser la rigueur.
   - **Groupe C** : un élève présente sa structure (fonctions, gestion d'erreurs). Valoriser la maîtrise.
3. (4 min) Conclusion : *un programme se planifie en pseudo-code, se teste avec un jeu d'essais et se corrige en distinguant le type d'erreur.* Annoncer la prochaine séance : page Web avec HTML.

6. Clôture : exit ticket — 110 – 120 min

Formulaire en ligne ou papier (annexe G) :

1. Un programme s'exécute mais affiche un mauvais résultat sans message d'erreur : de quel type d'erreur s'agit-il ?
2. Cite deux cas que tu testerais pour vérifier une calculatrice.
3. Auto-évaluation : *Je maîtrise / Je progresse / J'ai besoin d'aide.*

Lire les réponses avant la séance suivante pour ajuster les groupes de besoin.

Annexe A : Support résumé de la mini-leçon

Planifier, tester, déboguer Nom : _____ Date : _____

Idée → pseudo-code → code étape par étape → tests → corrections

Type d'erreur	Signe	Exemple
Syntaxe	Message avant l'exécution	`if x > 3` (deux-points oublié)
Exécution	Programme qui s'arrête (Traceback)	`10 / 0`
Logique	Résultat faux sans message	`moyenne = a + b / 2`

Modèle de calculatrice

```
while True:
    print("1 : addition   2 : soustraction   0 : quitter")
    choix = input("Choix : ")
    if choix == "0":
        break
    a = float(input("Premier nombre : "))
    b = float(input("Second nombre : "))
    if choix == "1":
        print("Resultat :", a + b)
    elif choix == "2":
        print("Resultat :", a - b)
```

Annexe B : Fiche de pratique guidée et grille du projet

Pratique guidée : mon programme

Nom : _____ Date : _____

3a. Pseudo-code de mon projet

Mon projet : ☐ Calculatrice ☐ Jeu de devinette

Étapes (à ranger dans l'ordre) :

3b. Jeu d'essais

Entrée	Résultat attendu	Résultat obtenu	Conforme ?

Grille d'évaluation du projet

Critère	Description	Points
Fonctionnement	Le programme fait ce qui est demandé	/5
Structure	Variables, condition et boucle utilisées à bon escient	/5
Tests	Jeu d'essais complet, erreurs corrigées	/5
Lisibilité	Commentaires, noms clairs, indentation	/5

Cartes d'aide à utiliser dans l'ordre

Aide 1 : regarde le pseudo-code du 3a et les modèles des séances précédentes.
Aide 2 : Quelle est la prochaine étape du pseudo-code ? Quelle structure faut-il ?
Aide 3 : demande à l'enseignant d'écrire la première ligne avec toi.

Annexe C : Niveau A (en dessous du niveau)

Niveau A : calculatrice à deux opérations

Nom : _____

Date : _____

Banque de mots : input, float, if, elif, print, +, -.

niveauA.py

```
a = _____(input("Premier nombre : "))
b = _____(input("Second nombre : "))
op = input("Operation (+ ou -) : ")
_____ op == "+":
    print("Resultat :", a _____ b)
_____ op == "-":
    print("Resultat :", a _____ b)
```

Activité : trois calculs

Calcul	Résultat attendu	Résultat obtenu
$5 + 3$		
$10 - 4$		
$2{,}5 + 1{,}5$		

Annexe D : Niveau B (au niveau)

Niveau B : calculatrice complète ou jeu Nom : _____
Date : _____

Option 1 : calculatrice. Menu en boucle avec les quatre opérations, option quitter, division par zéro refusée avec un message.
Option 2 : jeu de devinette. Nombre secret, indices « plus grand » ou « plus petit », compteur d'essais, rejouer.

Cas testé	Résultat attendu	Obtenu	Corrigé ?
Cas normal			
Cas limite (0, négatif)			
Saisie absurde (lettre)			

Bilan des erreurs rencontrées

Type d'erreur	Où ?	Comment corrigée ?

Annexe E : Niveau C (au-dessus du niveau)

Niveau C : projet structuré Nom : _____ Date : _____

Aide : fonction et gestion d'erreur

```
def addition(a, b):
    return a + b

try:
    a = float(input("Nombre : "))
except ValueError:
    print("Ce n'est pas un nombre.")
```

1. Ajoute une fonction par opération et utilise-les dans la boucle.
2. Garde un historique des calculs dans une liste (`historique.append(...)`) et affiche-le à la fin.
3. Explique pourquoi une fonction améliore la lisibilité : .....

Défi bonus : ajoute les puissances et le reste de la division (`**` et `%`).

Annexe F : Corrections et attendus (enseignant)

Annexe B

Pseudo-code attendu : boucle, menu, saisie de deux nombres, calcul selon l'opération, affichage, quitter. Barème : voir la grille.

Annexe C

`float`, `float`, `if`, `+`, `elif`, `-`. Résultats : 8, 6, 4.

Annexe D

Division par zéro : `if b == 0: print("Division impossible")`. Cas limites : 0, négatifs. Saisie absurde : `ValueError` (à gérer au niveau C). Les trois types d'erreurs doivent être nommés correctement.

Annexe E

Structure attendue : fonctions `addition`, `soustraction`, `multiplication`, `division` ; liste `historique` ; `try / except ValueError`.
Critères de réussite : programme qui ne plante pas sur une saisie absurde, fonctions utilisées, historique correct.

Annexe G : Exit ticket et grille d'observation

Exit ticket : programme complet Nom : _____ Date : _____

1. Un programme s'exécute mais affiche un mauvais résultat sans message d'erreur : de quel type d'erreur s'agit-il ?

2. Cite deux cas que tu testerais pour vérifier une calculatrice.

3. Je me sens : ☐ Je maîtrise ☐ Je progresse ☐ J'ai besoin d'aide

Attendus : 1. Une erreur de logique. 2. Par exemple une division par zéro, un nombre négatif, une saisie qui n'est pas un nombre.

Grille d'observation (enseignant)

Élève	Pseudo-code	Jeu d'essais	Erreurs corrigées	Groupe suivant